Beyond Static Thresholds: Context-Aware Correlation for Brute-Force Detection in a SIEM

A Reproducible Simulation Study with MITRE ATT&CK Mapping

Anuj Kumar
Master of Computer Applications (Cloud Computing & DevOps), University Institute of Computing, Chandigarh University
socanalystanuj@gmail.com | Guide: Name of Faculty

Abstract— Security teams commonly detect brute-force logons with a fixed threshold, such as "N failed logons (Windows Event 4625) in five minutes". Such rules are hard to tune: a low N floods the SOC with false alerts from typing mistakes, stale service credentials and shared VPN addresses, while a high N misses password-spraying and low-and-slow attacks. This paper studies the problem with a reproducible simulation of a 24-hour Windows domain (300 users, 40 attack campaigns per day, 30 independent days). Two static rules (per source address and per account) are compared with a context-aware set of rules that uses a baseline of each account's usual sources, a spraying rule, and a long-window rule. With an analyst budget of 10 false alerts per day, the best static rules detected 49% (per source) and 58% (per account) of the attack campaigns, whereas the context-aware rules detected 97% with 5.0 false alerts per day. An ablation shows which rule adds what, and a sensitivity test shows how the result degrades when the baseline is incomplete. The cost is a longer mean time to detect (18 min against 10 min). All data are synthetic; the findings are therefore a design guide to be validated on real logs, and the simulator is provided for reproduction.

Index Terms— SIEM, detection engineering, brute force, password spraying, false positives, MITRE ATT&CK, T1110, SOC operations

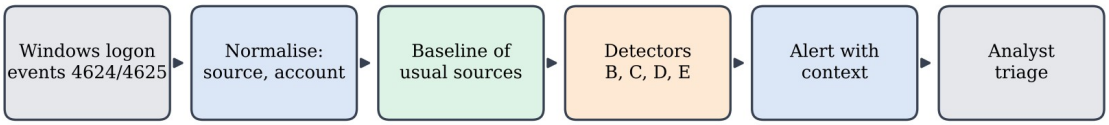


Fig. 1. Detection pipeline studied: logon events are normalised, enriched with a baseline of usual sources, passed to the detectors, and delivered to the analyst with context.

I. INTRODUCTION

A SIEM turns raw logs into alerts, but an alert is only as good as the rule behind it. The most common identity-attack rule is a count threshold on failed logons. Authentication failures are cheap to collect and map directly to MITRE ATT&CK technique T1110 (Brute Force) and its sub-techniques password guessing (T1110.001) and password spraying (T1110.003) [2]. Yet the same event also appears in normal work: a user mistypes a password, a service keeps an old credential, a vulnerability scanner probes accounts, or hundreds of remote users appear behind one NAT address. Because real attacks are rare compared with benign activity, even a rule with a low false-positive rate can produce mostly false alerts, the base-rate problem described by Axelsson [5]. Interviews with SOC analysts report that the large majority of alarms are false and that this erodes trust in the tools [7].

This paper asks a practical question: **how much do simple pieces of context (the usual source of an account, the number of distinct accounts hit by one source, and a longer time window) improve brute-force detection over a tuned static threshold, and at what cost?** The contributions are:

(1) a transparent simulator of a Windows logon environment with benign noise and four attack types; (2) a comparison of static and context-aware rules under a fixed false-alert budget, with an ablation and a sensitivity test; and (3) practical guidance for SOC detection engineers, including illustrative SPL.

II. BACKGROUND AND RELATED WORK

Windows records a failed logon as Event ID 4625 and a successful one as 4624 [4]. NIST SP 800-92 describes log management as the basis for security monitoring and incident investigation [1], and SP 800-61 gives the incident handling lifecycle that follows an alert [3]. MITRE ATT&CK provides a shared vocabulary of adversary techniques and recommends designing analytics around them [2][9]. Password spraying (one or two attempts against many accounts) and low-and-slow guessing are designed to stay below per-account lockout and count thresholds.

Evaluation of detectors is difficult. Sommer and Paxson note that anomaly detection in security faces a 'closed world' assumption and a high cost of errors [6], and Alahmadi et al. report that analysts regard most alarms as false [7]. These works motivate measuring detections by **false alerts per day**, which is the workload an analyst actually feels, in addition to recall.

III. METHOD

A. Simulated environment

One simulated day contains the events of a Windows domain. Benign behaviour includes successful logons, typing mistakes, users who forgot a password, a morning burst of failures from remote users behind one VPN address, services with stale credentials, and two scheduled vulnerability scans (Table I). A baseline holds, for each account, the sources from which it logged on in the previous days. A share of users (15% by default) sometimes log on

from a source that is not in the baseline, such as a new laptop or a home network.

TABLE I. Simulation parameters (per day)

Item	Value
Accounts / remote users / services	300 / 60 / 15 (3 with stale credentials)
Typing mistakes	Poisson, mean 0.8 episodes per user; 1–4 failures, then success
Forgotten-password users	12 per day, 5–12 failures, then success
VPN morning burst	30% of remote users, 1–3 failures from one NAT address
Vulnerability scans	2 per day, 40 accounts each, from one internal host
Users with unseen sources	15% (0–50% in the sensitivity test)
Attack campaigns	40: fast (12), short (6), spraying (12), low-and-slow (10)
Mean events / failures per day	6173 / 4540

B. Attack campaigns

Fast brute force: 40–300 guesses against one account at 1–5 s intervals. Short brute force: 6–15 guesses at 5–20 s. Password spraying: 25–120 accounts, 1–2 attempts each, 10–120 s apart. Low-and-slow: 15–40 guesses 2–7 minutes apart. 20% of campaigns come from an internal host; 15% of single-account campaigns (*stealth*) are run from the victim's own usual machine; 25% end with a successful logon.

C. Detection rules

R1 (per source): alert if one source address has at least N failures within 5 minutes. **R2** (per account): the same count for one account. The **context-aware** set is built in steps. **B:** a burst of at least 5 failures in 5 minutes for one (source, account) pair, suppressed when the source is in the account's baseline and the burst has 12 failures or fewer (typing mistakes). **C:** at least 15 distinct accounts failing from one source within an hour, suppressed if most of those accounts normally log on from that source (NAT, scanners are not in the baseline and remain alerts). **D:** at least 12 failures for one pair within 4 hours from a source outside the baseline (low-and-slow). **E:** a success after 5 or more failures from a new source (T1078 escalation). Alerts for the same key are merged for one hour.

D. Protocol and metrics

Each configuration was run on 30 independent simulated days (fixed seeds, 1000–1029). **Recall** is the share of the 40 campaigns that raised at least one alert for the attacker source (or targeted account) during the campaign. **False alerts per day** counts alerts not matching any campaign. **MTTD** is the mean time from the first guess to the first matching alert, over detected campaigns. Static thresholds N = 2…30 were swept, and the best N under a budget of 10 false alerts per day was chosen for each family.

IV. RESULTS

TABLE II. Main results (mean ± SD over 30 days)

Rule	Recall	False alerts/day	Alerts/ day	MTTD, min
R1, N = 3	0.84 ± 0.04	64.5 ± 5.9	114	7.1
R1, N = 5	0.75 ± 0.00	20.6 ± 1.5	65	2.3
R1, N = 9 (best, budget)	0.49 ± 0.05	8.6 ± 1.6	29	9.5
R2, N = 7 (best, budget)	0.58 ± 0.05	6.3 ± 2.0	27	13.1
Context-aware (B+C+D+E)	**0.97 ± 0.03**	**5.0 ± 1.4**	**60**	**18.4**

Under the budget of 10 false alerts per day, the best static rules detect only about half of the campaigns (49% and 58%). Loosening the threshold improves recall but at a high price: R1 with N = 3 detects 84% while raising 64 false alerts per day. The context-aware rules detect 97% of the campaigns with 5.0 false alerts per day (Fig. 2).

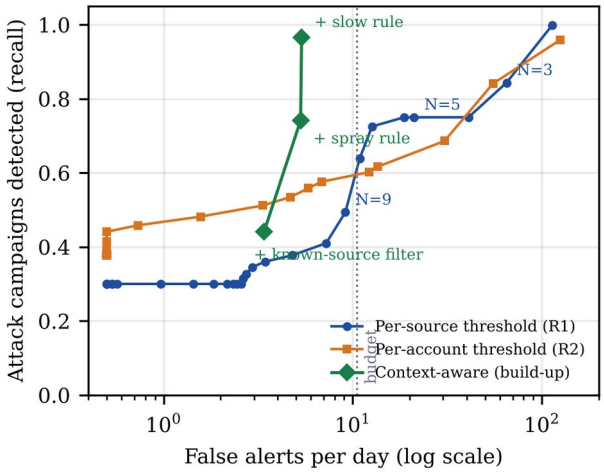


Fig. 2. Recall against false alerts per day. Static rules move along a trade-off curve; each context rule added moves the system up and to the left.

Fig. 3 shows why. All rules catch fast brute force. Static rules miss password spraying because each account sees only one or two failures, and they miss low-and-slow guessing because the count never reaches N inside five minutes. The context-aware rules cover these two types.

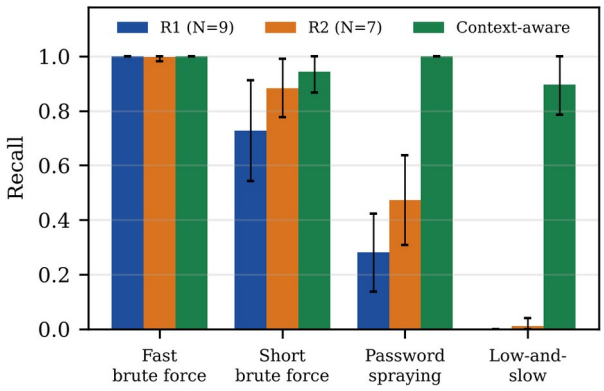


Fig. 3. Recall by attack type for the best static rules and the context-aware rules (error bars: SD over 30 days).

TABLE III. Ablation: effect of each context rule (cumulative)

Rules	Recall	FP/day	MTTD
B: pair burst + known-source filter	0.44	2.9	0.4
+ C: spraying rule	0.74	4.8	9.5
+ D: slow rule	0.97	4.9	18.4
+ E: success after failures	0.97	5.0	18.4

The ablation (Table III) shows that the known-source filter removes most false alerts of the plain pair burst (16.9 to 2.9 per day), the spraying rule adds 30 points of recall, and the slow rule adds 22 points. Rule E added no recall here because every compromised campaign had already raised an alert; its value is severity, since a success after failures indicates likely account takeover. The price is time: spraying and slow attacks need minutes to hours of evidence, so MTTD rises from 0.4 to 18.4 minutes.

TABLE IV. Sensitivity to an incomplete baseline

Unseen sources	Context recall	Context FP/day	R1 N=5 FP/day
0%	0.96	3.5	20.3
15%	0.96	5.1	20.8
30%	0.95	6.9	20.5
50%	0.96	8.8	20.1

If the baseline is incomplete, the context rules lose precision rather than recall: false alerts grow from 3.5 to 8.8 per day when half of the users sometimes log on from unseen sources (Table IV), still well below the static rule at the same recall level.

E. Case study (one simulated day, seed 1000)

A spraying campaign from one external address targeted **96 accounts** with 132 failures over 145 minutes. No account received more than 2 failures and the source never exceeded 8 failures in any five-minute window, so R1 (N = 9) and R2 (N = 7) stayed silent, while the context-aware rule C raised an alert because 96 distinct accounts failed from an address that none of them normally uses. On the same day the VPN gateway produced **51 failures across 24 accounts in one hour** (a normal Monday-style morning burst). R1 with N = 5 alerted three times on this address, while the context-aware rules stayed silent because most of those accounts normally log on through the gateway. The three misconfigured service hosts triggered R1 (N = 5) five times in total.

F. Validation on real Windows logs

To check the rules against genuine telemetry, we replayed them on the public OTRF Security-Datasets capture “PurpleSharp Active Directory Playbook I” (SDWIN-201022042947, ATT&CK T1110.003) [10] [11], which holds real Windows Security events from a domain lab. After filtering to events 4624/4625 it has 56 logons, of which 7 are failures against 7 different accounts from one workstation inside one second, a real password spray. The pair rule B and the per-account rule R2 raised 0 alerts, because each account failed only once. The per-source rule R1 (N=5) and the spraying rule C with M=5 or M=7 detected it; C with the M=15 used in the simulation raised 0 alerts and missed it (Table V).

TABLE V. Replay on the real spray (raw alerts, before de-duplication)

Rule	Real alerts	Hybrid detect
B pair, N=5	0	–
R2 account, N=5	0	–
R1 source, N=5	3	–
C spray, M=5	3	30/30
C spray, M=7	1	30/30
C spray, M=15	0	0/30

In a hybrid test, we injected this real campaign into the 30 simulated benign days: C with M=5 or M=7 found it every time at 1.9 false alerts/day, while M=15 never did. The lesson is practical: the distinct-accounts threshold must be set from the size of the attack one wants to catch, and the real campaign touched far fewer accounts than the simulated sprays. The dataset has no benign background, so it validates detection, not false-alert rates.

V. DISCUSSION

Why context works. A count on one key sees only one view of the data. Moving the key from the source to the (source, account) pair removes NAT noise; counting distinct accounts per source exposes spraying; a baseline of usual sources separates a typing mistake from a stranger. These checks mirror what an analyst does manually during triage, so encoding them gives the analyst a better alert and fewer cases to open.

Implementation. The rules are expressible in common SIEM query languages. Illustrative SPL for the pair burst and spraying rules is shown below (field names depend on the Windows add-on; not executed in this study):

```
index=wineventlog EventCode=4625 | bin _time span=5m
| stats count by _time, IpAddress, TargetUserName
| where count>=5
| lookup usual_sources TargetUserName, IpAddress OUTPUT known
| where isnull(known) OR count>12


index=wineventlog EventCode=4625 | bin _time span=60m
| stats dc(TargetUserName) AS accounts by _time, IpAddress
| where accounts>=15
```

Each alert should carry its ATT&CK mapping (T1110.001, T1110.003, T1078), the evidence count and the baseline result, so that triage starts with context [2][3].

TABLE VI. Mapping of the rules to ATT&CK and data needs

Rule	ATT&CK	Data needed
B: pair burst	T1110.001	4625: source, account, time
C: spraying	T1110.003	4625; baseline of usual sources
D: slow guessing	T1110.001	4625 over 4 h; baseline
E: success after failures	T1078, T1110	4624 and 4625, same source

Recommendations for SOC engineers. (1) Key detections on the pair (source, account), not on the source alone. (2) Maintain a baseline of usual sources and review its coverage, since coverage drives false alerts (Table IV). (3) Add a distinct-accounts rule for spraying and a multi-hour rule for slow guessing, and accept their longer detection time. (4) Keep a documented exception list for scanners and known misconfigured services instead of raising the threshold

for everybody. (5) Report false alerts per day and recall per attack type, not a single detection rate. (6) Re-tune after every change of environment, such as a new VPN or scanner.

Limitations. (i) The main experiment is simulated, and the same author designed both the attacks and the rules, which favours the rules; the only real log used is one short, attack-only capture (Section IV-F), so false-alert rates on real traffic remain unmeasured. (ii) Attackers are modelled with fixed patterns; an adversary who knows the baseline can imitate a usual source (the stealth campaigns already show this blind spot for low-and-slow attacks). (iii) Account lockout, MFA and other events (such as 4740) are not modelled. (iv) Recall is measured per campaign, and thresholds were chosen on the same simulation; a real deployment needs a separate validation period. (v) Windows logon types and Kerberos failures were not separated. These points are threats to validity; the numbers should be read as relative comparisons between rules, not as absolute performance.

VI. CONCLUSION AND FUTURE WORK

Static thresholds force a choice between missing attacks and flooding analysts. Adding three simple pieces of context (known sources, distinct accounts per source, and a long window) raised campaign recall from about 49% to 97% within a lower false-alert budget in the simulated environment, at the cost of later detection of slow attacks. The results are a design guide, not a measurement of real-world performance. Future work will replay the rules on larger public authentication datasets with benign background and on a Splunk or Wazuh home lab, add lockout and MFA events, learn the baseline automatically, and test adversaries that imitate usual behaviour.

Reproducibility. The simulator and analysis script (`siem_sim.py`, Python) generate all numbers and figures in this paper; seeds 1000–1029 are fixed. Code: GitHub link of the author.

REFERENCES

[1] K. Kent and M. Souppaya, *Guide to Computer Security Log Management*, NIST SP 800-92, 2006.

[2] MITRE, “Brute Force, Technique T1110,” *MITRE ATT&CK*. https://attack.mitre.org/techniques/T1110/

[3] P. Cichonski, T. Millar, T. Grance and K. Scarfone, *Computer Security Incident Handling Guide*, NIST SP 800-61 Rev. 2, 2012.

[4] Microsoft, “4625(F): An account failed to log on,” *Windows security auditing reference*, Microsoft Learn.

[5] S. Axelsson, “The base-rate fallacy and the difficulty of intrusion detection,” *ACM Trans. Inf. Syst. Secur.*, vol. 3, no. 3, pp. 186–205, 2000.

[6] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *Proc. IEEE Symp. Security and Privacy*, 2010, pp. 305–316.

[7] B. A. Alahmadi, L. Axon and I. Martinovic, “99% false positives: A qualitative study of SOC analysts’ perspectives on security alarms,” in *Proc. USENIX Security Symp.*, 2022.

[8] Splunk, “Search Reference: stats, bin, lookup,” *Splunk Documentation*.

[9] B. E. Strom et al., *MITRE ATT&CK: Design and Philosophy*, MITRE Technical Report, 2018 (rev. 2020).

[10] R. Rodriguez and J. Rodriguez, “Security-Datasets (Mordor),” Open Threat Research Forge. https://github.com/OTRF/Security-Datasets

[11] OTRF, “PurpleSharp Active Directory Playbook I, dataset SDWIN-201022042947,” Security-Datasets, 2020.